EHM(X)：構文・定義・定理のステートメント

原典との対応と，承認済みの拡張 / 2026 年 10 月 16 日

範囲と状態

定理 7・16・17・系 7 は未証明である．`Statements.lean` の宣言は命題を定義する `def ... : Prop` であり，証明でも公理でもない．今回の対象は構文・定義・条件・仮定・定理文の修正である．旧 `Constraints`，独立した意味論的代入作用，旧四定理，旧正規化と依存する具体 solver は削除した．

参照は Martin Sulzmann, *A General Framework for Hindley/Milner Type Systems with Constraints*, 2000, §§3–4, Appendix C.4 である．原典公開ページ．

原典・対象	Lean の対応
定義 2–5	`ConstraintTheory`：含意，射影，自由変数，充足可能性．
§3.2，定義 6–7	`TypeSignature`，`TypeTerm`，`Substitution`，`TermConstraintTheory`．
§3.4，定義 14–18	スキーム同値，環境比較，整合性，`Generalizes`，`Scheme.Approximates`．
図 3.1 と図 3.4	`Presentation.term/constraint`．解決済み集合 S とは別の添字．
定義 20，22–25	構文的な代入域，制約問題，全変数での主要性．
定義 26–28，注 3	型の等式自由性，有限述語集合，有効な符号化，代表元の条件．
図 4.4	帰納的関係 `Generates` と `Reconstructs`，別に実装 `generate` と `inferTerm`．
定理 7，16，17，系 7	原典に対応する関係の命題，EHMX 版，実装についての命題を分離．
定義 29	`Statements.PrincipalPair`．計算結果の主要性とは別定義．

維持する差異

1. 効果の名前と潜在効果，有限集合の効果項，`perform`，handler，値制限．
2. 正規形比較は対象残余の下で一致を要求する．因子は全評価に共通の一つの有限代入．
3. 定理 17 の環境・効果の一致も対象制約 C の下で述べる．
4. 比較の全称右規則は結論 C を維持する．型付けの全称導入は射影のガードを維持する．
5. 比較導出の各節点の解決済み条件を明示する（原典付録 197 ページの条件）．
6. EHMX 版で使う lifting を明示し，等式自由性は型項だけに課して効果等式を許す．

`existsIntro` 自体は図 3.4 に存在する．これを EHMX 独自規則とは呼ばない．式変数の de Bruijn 添字，有限保存範囲，明示的な同値関係は Lean の表現上の選択である．

1　制約系，型項，有限同時代入

`ConstraintTheory` の含意 $C \vdash_X D$ は独立した関係であり，一つの固定した解釈で定義しない．$\equiv_X$ は双方向の含意．反射性・推移性，$\top$，連言の導入・除去に加えて次を満たす．

$$C \vdash_X \exists A.C, \quad C \vdash_X D \Longrightarrow \exists A.C \vdash_X \exists A.D,$$
$$\exists A.(C \wedge \exists A.D) \equiv_X (\exists A.C) \wedge (\exists A.D),$$
$$\exists A.\exists B.C \equiv_X \exists B.\exists A.C.$$

空集合と集合の和による射影は有限反復を表す．保存範囲外への射影は制約を変えない．

$$\mathrm{FV}(C) = \{a \mid \exists a.C \not\equiv_X C\}, \quad \mathrm{Sat}(C) \iff \top \vdash_X \exists \mathrm{FV}(C).C, \quad \mathrm{Adm}_S(C) \iff \exists D \in S.\ C \equiv_X D.$$

`SolvedForms S` は $S \subseteq \mathrm{Sat}$ のみ．射影閉性や改名閉性は別命題であり，一律の連言閉性は要求しない．空の S を定義で排除しない．

開いた型項と述語

`TypeSignature` は任意の構成子と型・効果引数の個数を指定する．`TypeTerm` は変数，効果付き矢印，その構成子からなる自由項代数．`TypeSyntax` はこれとの同型と，構文操作・自由変数の対応を持つ．閉じた型項の存在は要求しない．整数・真偽値などの定数の型は有限初期環境が与える．

$$p ::= P(\bar{t}; \bar{\epsilon}) \mid t = u \mid t \preceq u \mid \epsilon = \eta \mid \epsilon \subseteq \eta \mid \mathrm{lacks}(o, \epsilon),$$
$$C ::= \top \mid p \mid C \wedge C \mid \exists A.C \mid \forall B.C.$$

全称射影と効果述語は EHMX 拡張．定義 2 の有限トークン集合は `ConstraintSyntax.TokenSet` とその連言として表せる．`ConstraintPresentation` は全制約を同値な構文に表すが，その符号化に連言などの**構文的一致**を要求しない．有限集合の連言を排除しないためである．

代入は構文から一度だけ構成する

`FiniteSubstitution` は有限写像，`Substitution` はそのうち冪等なもの（定義 6）．`Renaming` は有限置換．写像の合成や制限には冪等性を自動付与しない．

$$\mathrm{dom}(\theta) = \{a \mid \theta a \neq a\} \quad \text{（構文的な不一致）}, \qquad \mathrm{cod}(\theta) = \mathrm{FV}\{\theta a \mid a \in \mathrm{dom}(\theta)\}.$$

保存範囲 `supportBound` は代入域とは異なる．述語引数には同時代入し，量化の下では束縛名を新鮮化してから再帰する．`Theory.subst` はこの構文的作用だけを使う．

D1–D5 と，**述語 p に対する** D6 を条件に置く．

$$[t/a]p \equiv_X \exists a.(p \wedge a = t) \quad (a \notin \mathrm{FV}(t)).$$

D4 の異なる変数という慣習は $i \neq j$ と明記する．一般の制約への代入による含意保存・同値保存・表現独立性は `SubstitutionPreservesEntailment` 等の**未証明の命題**とし，追加の仮定にしない．型比較の REFL・ASYM・TRANS・CONTRA と効果に関する法則は `Theory` に記述する．

2　スキーム，環境，比較，一般化

$$s ::= t \mid \forall A.C \Rightarrow t, \qquad \mathrm{FV}(\forall A.C \Rightarrow t) = (\mathrm{FV}(C) \cup \mathrm{FV}(t)) \setminus A.$$

図 3.1 は一層のスキームを使う（§3.4.5）．`Scheme.Equivalent` は束縛名の改名と条件の論理的同値を含み，型付けと比較にも同値の移送規則を置く．環境は，重複のない有限定数表と局所変数のリストである．自由変数，代入，許容性，環境比較は**両方**に作用する．

比較関係 i

$$\frac{C \vdash_X t \preceq u}{C \vdash_i t \preceq u} \qquad \frac{\mathrm{dom}(\theta) \subseteq A \quad C \vdash_X \theta D \quad C \vdash_i \theta t \preceq s}{C \vdash_i (\forall A.D \Rightarrow t) \preceq s}$$

$$\frac{A \cap \mathrm{FV}(C, s) = \varnothing \quad C \wedge D \vdash_i s \preceq t}{C \vdash_i s \preceq (\forall A.D \Rightarrow t)}.$$

左規則の右辺 s は任意のスキーム．右規則の結論 C は承認済みの変更である．`Instance` はこの導出の存在．`SolvedInstance` は全節点の判断制約が Adm_S に属する導出の存在で，右規則の $C \wedge D$ も検査する．定義 14 の環境比較は同じ変数・定数ごとの比較，定義 15 の制約付き環境比較は $C \vdash_X D$ と $C \vdash_i \Gamma \preceq \Gamma'$ の組である．

定義 17 の整合性は, 代入後の制約・スキーム・環境が許容されること. `ConsistentConstraint`, `Scheme.Consistent`, `Context.Consistent` に分ける．スキームへの代入では先に束縛を新鮮化する．S の任意代入閉性は課さない．

定義 18：近似

$C \vdash s \mapsto_S s'$ は，s が Sat で許容され，C と s' が S で許容され，

$$\top \vdash_i s \preceq s', \qquad \forall u \text{ 許容}.\ (C \vdash_i s \preceq u)_{\mathrm{solved}} \Longrightarrow (C \vdash_i s' \preceq u)_{\mathrm{solved}}$$

を満たすこと．最初の比較は無条件である．環境近似は各要素の近似．実現可能性は $\exists t.\ C \vdash_i s \preceq t$ で，ground 値の表現可能性ではない．

§3.4.4：一般化関係

`Generalizes S C Γ t Co so` を $\mathcal{G}_S(C, \Gamma, t; C_o, s_o)$ と書く．C，C_o，s_o が許容され，ある A，C_1，C_2 について

$$C_o \equiv_X \exists A.C, \qquad C_o \equiv_X C_1 \wedge \exists A.C_2,$$
$$s_o \equiv \forall A.C_2 \Rightarrow t, \qquad A \cap \mathrm{FV}(C_1, \Gamma) = \varnothing.$$

最大の A を返す関数に固定しない．原典にない $C \equiv_X C_1 \wedge C_2$ を追加しない．閉環境，閉スキーム，閉判断は別定義であり，環境が閉じているだけでは判断全体は閉じない．

3　二つの型付け形式

`Derivation p fragment` の p は `term` または `constraint`．`Fragment.core/handlers` は操作規則を使えるかを指定する．**どちらの添字も導出全体に共通**であり，入力だけを制限しない．解決済み集合 S はこの二つとは独立である．`constraint` 形式の型の位置には変数と矢印だけを許す．制約内の述語引数は任意の型項を許す．

式は変数，定数，ラムダ，適用，let，perform，handler．値は変数・定数・ラムダ．判断 $C;\Gamma \vdash e : s \,!\, \epsilon$ は以下の規則と許容性検査で定める．

$$\frac{\Gamma(x) = s}{C;\Gamma \vdash x : s \,!\, \varnothing} \qquad \frac{C;t :: \Gamma \vdash e : u \,!\, \epsilon}{C;\Gamma \vdash \lambda e : t \xrightarrow{\epsilon} u \,!\, \varnothing}$$

定数も初期環境の検索によって同じ形で型付けする．

$$\frac{C;\Gamma \vdash f : t \xrightarrow{\eta} u \,!\, \epsilon_f \quad C;\Gamma \vdash x : t \,!\, \epsilon_x}{C;\Gamma \vdash f\ x : u \,!\, \epsilon_f \cup \epsilon_x \cup \eta}$$

$$\frac{v \text{ は値} \quad C;\Gamma \vdash v : s \,!\, \varnothing \quad C;s :: \Gamma \vdash e : t \,!\, \epsilon}{C;\Gamma \vdash \mathbf{let}\ v\ \mathbf{in}\ e : t \,!\, \epsilon}$$

非値の let は単相型 t で束縛し，二つの即時効果を合併する．Sub は型比較だけを行う．

$$\frac{e \text{ は値} \quad A \cap \mathrm{FV}(C,\Gamma) = \varnothing \quad C \wedge D;\Gamma \vdash e : t \,!\, \varnothing}{C \wedge \exists A.D;\Gamma \vdash e : \forall A.D \Rightarrow t \,!\, \varnothing}$$

全称除去を分ける

term 形式の `forallElimSubst`：

$$\frac{C;\Gamma \vdash e : \forall A.D \Rightarrow t \,!\, \epsilon \quad \mathrm{dom}(\theta) \subseteq A \quad C \vdash_X \theta D}{C;\Gamma \vdash e : \theta t \,!\, \epsilon}.$$

constraint 形式の `forallElimConstraint`（図 3.4）：

$$\frac{C;\Gamma \vdash e : \forall A.D \Rightarrow t \,!\, \epsilon \quad C \vdash_X \exists A.D}{C \wedge D;\Gamma \vdash e : t \,!\, \epsilon}.$$

同じ constraint 形式の `existsIntro`：

$$\frac{A \cap \mathrm{FV}(\Gamma, s, \epsilon) = \varnothing \quad C;\Gamma \vdash e : s \,!\, \epsilon}{\exists A.C;\Gamma \vdash e : s \,!\, \epsilon}.$$

最後の効果の非脱出は EHMX の追加条件．s は qualified スキームでもよい．

制約の弱化，スキーム・環境の同値移送，論理的な効果等式による変更も持つ．`Derivation.Valid` は**各節点**の制約・環境・結論スキームの許容性と，型項の提示形式を検査する．両形式の対応は `presentationCorrespondence` という別の未証明命題であり，S を Sat に選ぶことと混同しない．

4　EHMX の操作と handler

操作宣言 $o : \forall B.\ t_{\mathrm{in}} \Rightarrow t_{\mathrm{out}}$ は $\mathrm{FV}(t_{\mathrm{in}}, t_{\mathrm{out}}) \subseteq B$ を満たす.

$$\frac{\mathrm{dom}(\theta) \subseteq B \quad C; \Gamma \vdash e : \theta t_{\mathrm{in}} \,!\, \epsilon}{C; \Gamma \vdash \mathbf{perform}\ o\ e : \theta t_{\mathrm{out}} \,!\, \{o\} \cup \epsilon}.$$

`Operation.FreshInstance` は宣言の束縛名の単射な改名を指定する．handler の本体・戻り節・操作節は

$$C; \Gamma \vdash e : t_b \,!\, \epsilon_b, \qquad C; t_b :: \Gamma \vdash r : t_r \,!\, \epsilon_r,$$
$$D; (t_{\mathrm{out}} \xrightarrow{\epsilon_o} t_r) :: t_{\mathrm{in}} :: \Gamma \vdash h : t_h \,!\, \epsilon_h.$$

操作節の継続は第 0 変数，操作引数は第 1 変数．handler の条件は

$$C \vdash_X \forall B. \exists W. (D \wedge t_h \preceq t_r \wedge \epsilon_h \subseteq \epsilon_o),$$
$$C \vdash_X \mathrm{split}(o, \epsilon_b, \rho), \quad C \vdash_X \rho \subseteq \epsilon_o, \quad C \vdash_X \epsilon_r \subseteq \epsilon_o,$$

ここで

$$\mathrm{split}(o, \epsilon_b, \rho) = (\rho \subseteq \epsilon_b) \wedge (\epsilon_b \subseteq \{o\} \cup \rho) \wedge \mathrm{lacks}(o, \rho).$$

結論は $C; \Gamma \vdash \mathbf{handle}\ e\ o\ r\ h : t_r \,!\, \epsilon_o$．型の Sub と効果の任意拡大を一つの規則にはしない.

名前の条件

`Operation.Scoped` は次を明示する.

- B は環境，本体・結果型，外側の即時効果から新鮮である.
- W は環境，本体・結果型，操作入出力型，B から新鮮である.
- W は外側の制約 C と即時効果に自由出現しない.

宣言が閉じていることとこれらの条件は別である．操作節自身の導出にも `Derivation.Valid` を再帰適用し，その条件 D の許容性を検査する.

一般化した計算結果

`ComputationScheme` は $R = (A, C, t, \epsilon)$．型部分は $\forall A. C \Rightarrow t$，外側の制約は $\exists A. C$．A を最大集合に固定する関数は設けない.

$$R\ \mathrm{FiniteInstance}\ (D, u, \eta) \iff \exists \theta.\ \mathrm{dom}(\theta) \subseteq A \wedge D \vdash_X \theta C \\ \wedge D \vdash_X \theta t \preceq u \wedge D \vdash_X \theta \epsilon = \eta.$$

同じ一つの有限代入を使う．比較は従来どおり対象制約 D の下で行う．`PrincipalComputationScheme` は，条件 C の下での単相型付けと環境への非脱出を持ち，全ての単相型・効果の競合結果をこの関係で包含すること．これは非値の let を多相にする規則ではなく，系 7 の原典に対応する結論とも別の命題である.

5　図 4.4：制約生成と再構成

`Generates` は帰納的関係．$\Gamma \vdash e \rightsquigarrow_i (t, \epsilon; C)$ と書く．結果型は常に新鮮な型変数 $t = \alpha$．以下の内部名は環境・出力から新鮮で，同じ規則内の別の型変数・効果変数はそれぞれ異なる．表示では $\top$ との連言を省略する（Lean の補助関数 `checked` はこれを明記する）．

規則	子の推論と生成制約
変数・定数	環境の $\forall A.D \Rightarrow t$ を新鮮化し，$\exists A.(D \wedge t = \alpha \wedge \varnothing = \epsilon)$．単相なら射影なし．
ラムダ	子は $a :: \Gamma \vdash e \rightsquigarrow_i (b, \mu; D)$．$\exists a, b, \mu.(D \wedge (a \xrightarrow{\mu} b) = \alpha \wedge \varnothing = \epsilon)$．
適用	子は $(a, \mu; C_f)$ と $(b, \nu; C_x)$．$\exists a, b, \mu, \nu, \rho.(C_f \wedge C_x \wedge a \preceq b \xrightarrow{\rho} \alpha \wedge \mu \cup \nu \cup \rho = \epsilon)$．
値 let	値の子は $(a, \varnothing; C_v)$．$\forall a.C_v \Rightarrow a$ を束縛して本体を推論し，結果は $(\exists a.C_v) \wedge C_b$．内部で既に射影した変数を全称一般化しない．
非値 let	単相 a を束縛．子は $(a, \mu; C_v)$ と $(\alpha, \nu; C_b)$．$\exists a, \mu, \nu.(C_v \wedge C_b \wedge \mu \cup \nu = \epsilon)$．
Equ	$C \equiv_X D$ なら生成結果の C を D に変更する．段階を限定しない．

perform の子には要求型 t_{in} と別の新鮮型 a を与え，$C_a \wedge a \preceq t_{\text{in}} \wedge t_{\text{out}} = \alpha \wedge \{o\} \cup \mu = \epsilon$ を生成する．宣言の新鮮名と a，μ は射影する．handler は本体型 a，結果型 b，戻り節型 r，操作節型 k を区別する．生成条件には $r \preceq b$ と，$\forall B.\exists k, \omega.(C_h \wedge k \preceq b \wedge \omega \subseteq \epsilon_o)$ を含む．本体・結果・戻り節と外側の補助効果は外で射影する．子の型と要求型を同一にして自明な比較に置き換えない．

再構成関係と実装を分ける

`Reconstructs.finish` は生成制約 C と恒等代入の問題を主要正規化し，出力 (D, θ) から次を返す．

$$\psi = \theta|_{\text{FV}(\Gamma)}, \qquad D, \quad \theta\alpha, \quad \theta\epsilon.$$

帰納的関係は特定の計算関数を呼ばない．`generate` は保存範囲の上方から新鮮名を割り当てる一つの戦略で，値 let の条件に選んだ同値簡約器 N を適用する．`generateProjected` は出力外の名前を隠す．`inferTerm` は最後に一度だけ，同じ `PrincipalNormalizer` P を呼ぶ．変数・操作がなければ `none`，正規形がなければ `unrepresented` を返す．

この実装と帰納的関係の対応は `implementationGeneration` 等の未証明課題．関係の定理を記述しただけで，実装の定理が得られたとはしない．

6　正規形と定義 28

制約問題 $p = (C, \sigma)$ は冪等な有限代入を持ち，$\sigma C \equiv_X C$ を満たす．入力と出力に同じ型 `ConstraintProblem` を使う．

$$n \succeq m \iff \exists \phi.\, \forall a.\, C_m \vdash_X (\phi \circ \theta_n)a = \theta_m a \\ \wedge\, C_m \vdash_X \phi C_n.$$

等式は型・効果の各 sort で述べる．ϕ は有限・冪等で，一つを全変数・全評価に使う．**対象残余 C_m の下で一致する点は原典からの承認済み変更**である．

$$\mathrm{Normal}_S(p, n) \iff p, n \text{ が問題} \wedge C_n \in S \wedge p \succeq n,$$
$$\mathrm{Principal}_S(p, n) \iff \mathrm{Normal}_S(p, n) \wedge \forall m.\, \mathrm{Normal}_S(p, m) \Longrightarrow n \succeq m.$$

`PrincipalNormalForms`（定義 25）は「正規形があれば主要正規形がある」．`PrincipalNormalizer` は実際の関数と，成功の主要性，失敗と正規形不在の同値を持つ．`ofExistence` は古典選択による非計算的な実装で，計算可能性は主張しない．

四条件は分けて記述する

1. 定義 28(i)：`ProjectionFreeProcedure`．入力は有限の原始述語集合と代入（`PrimitiveProblem`）．自然数コードから入力・出力を復号する `SolverEncoding` と，計算可能なコード上の関数を持つ．全入力の符号化可能性，成功時の主要性，失敗条件を要求する．抽象的な有限写像全体に `Primcodable` を仮定しない．
2. 定義 28(ii)：`AllTypeEquationFree`．文字どおり

   $$C \in S \iff \mathrm{Sat}(C) \wedge \forall t, u.(C \vdash_X t = u \Longrightarrow \top \vdash_X t = u).$$

   型項だけに適用し，効果の等式は許す．注 3 の代表元方式は別の `EquationFreeRepresentatives`：上の左辺を $\mathrm{Adm}_S(C)$ に置き換え，同値な S の要素の構文的一意性を加える．両方式を同一視しない．
3. 定義 28(iii)：`ExactResidual`．**全ての**正規形 (D, θ) について $D \equiv_X \theta C$．選ばれた一出力だけではない．
4. 定義 28(iv)：`Lifting`.

   $$\exists A.C \in S \Longrightarrow \exists \theta.\, \mathrm{dom}(\theta) \subseteq A \wedge \theta C \in S.$$

`AdmissibleLifting` は最後の両方の所属を Adm_S に換えた別条件．EHMX の命題で使うのはこちらであり，代表元に関する条件と自動的に同じとは扱わない．`LiftingProperty` は四条件の組だが，四つの型推論定理全体の共通仮定にはしない．

7　四定理のステートメント

以下はすべて `Statements` 内の未証明命題．HMX 対応の主張にも承認済みの値制限と効果・比較の変更が残る．core は操作規則を含まず，handlers はそれを含む．

定理 7：導出変換

`Transformation p fragment` の二つの結論は，同じ p と fragment を使う．入力は S を Sat とした型付け．$C \in \mathrm{Adm}_S$ と環境近似 $C \vdash \Gamma \mapsto_S \Gamma'$ の下で

$$C;\Gamma \vdash e : t \,!\, \epsilon \Longrightarrow C;\Gamma' \vdash e : t \,!\, \epsilon,$$
$$C;\Gamma \vdash e : s \,!\, \epsilon \wedge s \text{ が C で実現可能} \Longrightarrow \exists s'.\ C;\Gamma' \vdash e : s' \,!\, \epsilon \wedge C \vdash s \mapsto_S s'.$$

`theorem7` は $S \subseteq \mathrm{Sat}$，主要正規形の存在，射影閉性を前提に term/core の変換を主張する．`theorem7EHMX` は許容版 lifting と閉じた操作宣言を加え，両 p の handlers の変換を主張する．提示形式を S の選択と混同しない．

定理 16：健全性

再構成関係が (ψ, D, t, ϵ) を返し，入力環境が許容されるなら

$$D;\psi\Gamma \vdash e : t \,!\, \epsilon, \quad \psi D \equiv_X D, \quad \psi t = t, \quad \psi\epsilon = \epsilon.$$

入力環境の $\top$ での実現可能性を加えない．出力の固定性は結論である．`theorem16` は $S \subseteq \mathrm{Sat}$，主要正規形，射影閉性を前提とする core 版．`theorem16EHMX` は許容版 lifting と閉じた操作宣言を追加する．

定理 17：完全性

$C;\sigma\Gamma \vdash e : s \,!\, \epsilon$ と $\sigma\Gamma$ の C での実現可能性から，再構成結果 (ψ, D, t, η) と一般化結果 C_o, s_o が存在し，

$$\mathcal{G}_S(D, \psi\Gamma, t; C_o, s_o), \qquad \exists\phi.$$
$$C \vdash_X (\phi \circ \psi)|_{\mathrm{FV}(\Gamma)} = \sigma|_{\mathrm{FV}(\Gamma)}, \quad (C \vdash_i \phi s_o \preceq s)_{\mathrm{solved}},$$
$$C \vdash_X \phi C_o, \qquad C \vdash_X \phi\eta = \epsilon.$$

環境の一致は各自由変数についての型・効果等式．全てを同じ因子 ϕ が満たす．`theorem17` の前提は $S \subseteq \mathrm{Sat}$ と主要正規形．射影・改名の全体閉性を加えない．`theorem17EHMX` は許容版 lifting と閉じた操作宣言を追加する．

系 7：閉環境の結果

$\mathrm{FV}(\Gamma) = \varnothing$, Γ が $\top$ で実現可能, $\top;\Gamma \vdash e : s!\epsilon$ なら, 再構成結果と $\mathcal{G}_S(D, \psi\Gamma, t; \top, s_o)$ が存在し, $(\top \vdash_i s_o \preceq s)_{\mathrm{solved}}$ と $\top \vdash_X \eta = \epsilon$ を満たす．`corollary7` と `corollary7EHMX` の前提はそれぞれ定理 17 の対応版と同じ．これは一般化の外側を $\top$ にする原典対応の結論である．

8　実装についての命題と，残る証明

同じ簡約器・正規化器を使う

`implementationSoundness`，`implementationCompleteness`，`implementationCorollary7` は全て**同じ引数 N と P** に対する命題である．各命題は再構成関係の存在を `inferTerm sig X N P 「 e = some ...` に置き換え，出力 n から環境代入，型，効果を読む．一般化は関係のままである．成功時に別の正規化器へすり替えない．主要正規形の存在は P の成功・失敗仕様に含まれる．

`implementationGeneration` は実装の生成出力を `Generates` へ接続する命題．完全性には逆方向に必要な十分性も確認する必要があり，この一方向だけで済んだとはしない．`implementationPrincipalComputation` は成功出力に対応する計算スキームの主要性と，値なら主要な値スキームであることを主張する**別の未証明命題**．

定義 29 と EHMX の主要性を区別する

`PrincipalPair 「 e C s ε` は

$$C;\Gamma \vdash e : s \,!\, \epsilon \ \wedge\ \forall D, u.\ D;\Gamma \vdash e : u \,!\, \epsilon \Longrightarrow (D \vdash_X C \wedge D \vdash_i s \preceq u).$$

`PrincipalValue` は値，$\top$ での型付け，任意の許容な競合型への解決済み比較を要求する．

`PrincipalComputationScheme` は即時効果も含めた `FiniteInstance` による主要性を要求する．これらを一つの定義にまとめない．

検査の範囲

Lean の全体ビルドは命題の型と依存関係を検査する．`DefinitionChecks` は型・効果の捕獲回避，入れ子の量化，述語引数・潜在効果への代入と，新しい生成規則・多相 let・handler のインターフェースを検査する．`LogicalTheoryChecks.no_closed_type` は，閉型がなくても型構文を与えられる例を確認する．これらは定理 7・16・17・系 7 の証明ではない．

証明を再開する順序

1. 述語への D6 と論理法則から，構成した代入の同値・含意保存と表現独立性を導く．これらを新しい仮定として追加しない．
2. スキーム比較の移送と，定理 7 の一般化・多相 let の近似を構成する．無条件の最初の比較と，解決済み比較の最大性を区別する．
3. 定理 7 と EHMX 版を証明する．constraint 形式と term 形式の対応，handler の量化と非脱出を別々に確認する．
4. 定理 16，定理 17，系 7 の順に関係の命題を証明し，同じ N・P を使う実装の命題へ接続する．
5. その後に Haskell 型クラスの具体的な X と solver を構築する．

証明が閉じない場合は原因と必要条件を記録する．結論を仮定に移したり，型検査を「定理が証明された」と報告したりしない．